```
template <int M, int N>
int countPathsWithCost(int (&cost)[M][N], int i, int j, int remaining){
    // Error Check: If the current position is out of bounds, return 0
    if (i >= M || j >= N)
        return 0;

    // Subtract the cost of the current cell from the remaining cost
    remaining -= cost[i][j];

    // If the remaining cost is negative, return 0 as this path cannot reach the
last cell
    if (remaining < 0)
        return 0;

    // If we have reached the last cell, check if the remaining cost is exactly 0
    if (i == M - 1 && j == N - 1) {
    //return 1 if the remaining cost is exactly 0, otherwise return 0
        return (remaining == 0) ? 1 : 0;
    }

    // Recursively count the paths by moving right and down
    return countPathsWithCost(cost, i, j+1, remaining) + countPathsWithCost(cost,
i+1, j, remaining);
}

/*
PSEUDOCODE:
countPathsWithCost(matrix, row, col, remainingCost):
    if the initial position is out of bounds:
        return 0

    Each function call:
        subtract the cost of the current cell from remainingCost

    If remainingCost is negative:
        return 0

    If we have reached the last cell:
        if remainingCost is 0:
            return 1
        else:
            return 0

    return countPathsWithCost(matrix, row, col + 1, remainingCost) +
countPathsWithCost(matrix, row + 1, col, remainingCost)
*/
```